

bok25

基
礎
班

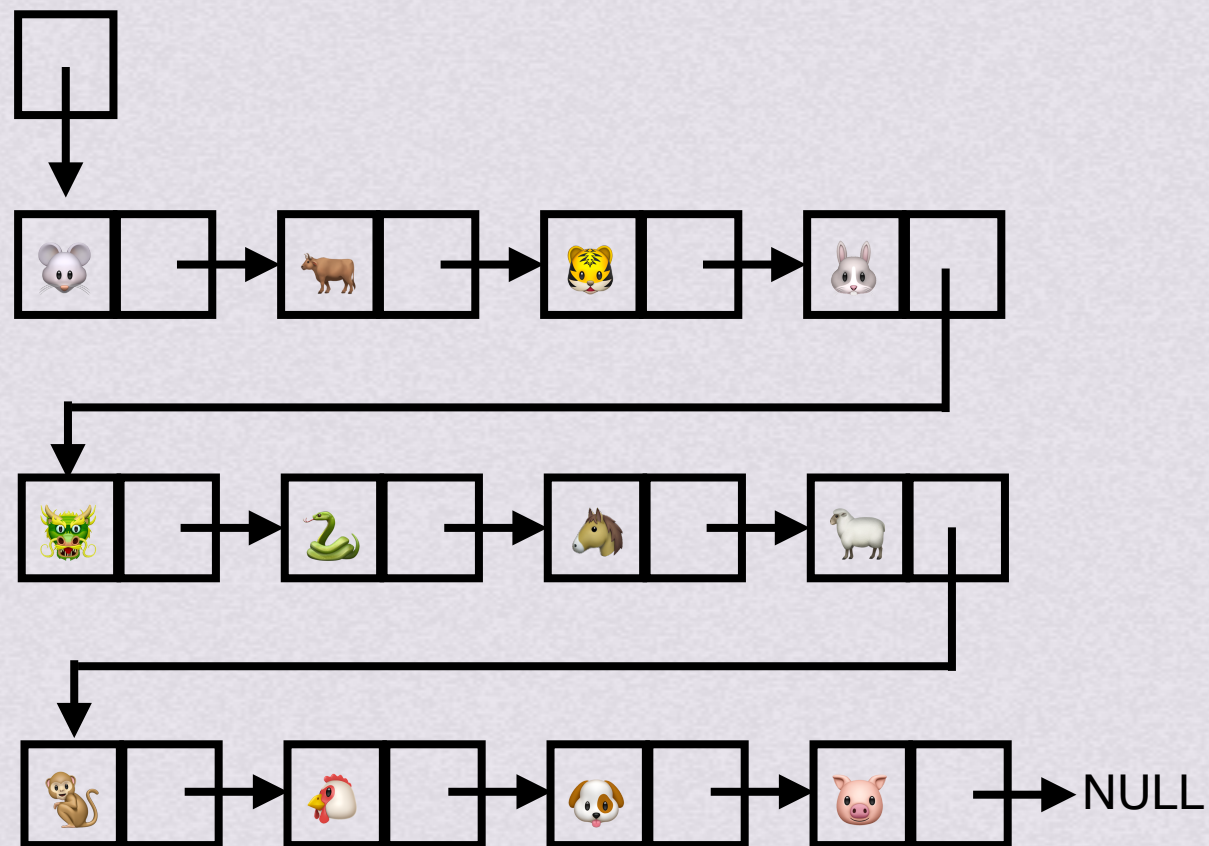
、**考研数据結構**



链表是什么？

(单链表为例)

头节点

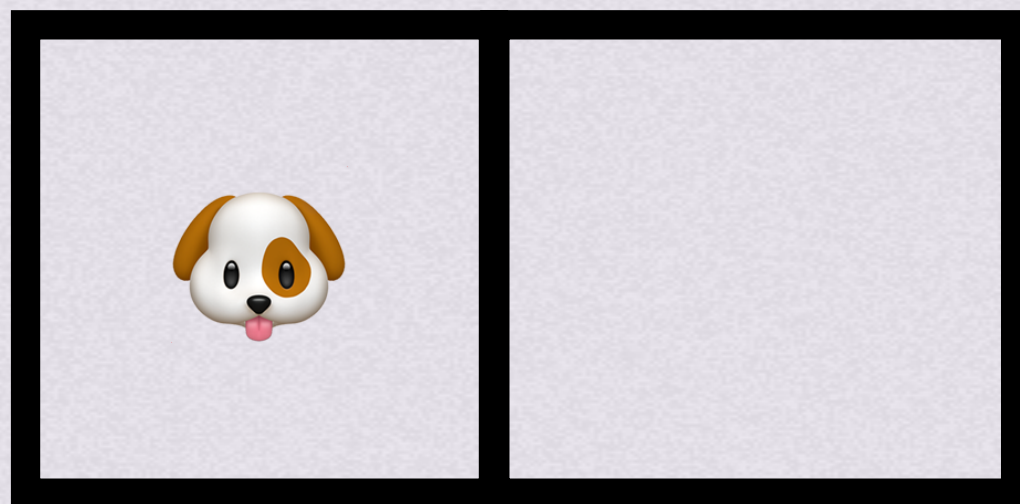


在内存中的分布

离散分布



链表是什么？（单链表为例）



数据域

指针域



链表是什么？（单链表为例）

代码呈现

```
//-----单链表的存储结构-----
```

```
typedef struct LNode
```

```
{
```

```
ElemType data;
```

```
//节点的数据域 (🐶)
```

```
struct LNode *next;
```

```
//节点的指针域（指向下一数据）
```

```
}LNode,*LinkList;
```




链表是什么？ 两种名称的区别

```
//- - - - - 顺序表的存储结构- - - - -  
typedef struct{  
    ElemType *elem;    //存储空间的基地址  
    int length;        //当前长度  
}SqList;              // 顺序表的结构类型为 SqList
```

```
//-----单链表的存储结构-----  
typedef struct LNode  
{  
    ElemType data;      //节点的数据域 (👉)  
    struct LNode *next; //节点的指针域 (指向下一数据)  
}LNode,*LinkedList;
```

LNode *LinkedList
↓ 申请链表结构体节点 ↓

LNode *p LinkedList p

若定义LinkedList p和LNode *p

p

p:指向某节点的指针变量
表示该节点的地址

*p

*p:对应的结点变量
表示该结点的名称



链表是什么？ 两种名称的区别

LinkedList L中， L是单链表头节点的指针变量， L是一个指针

LNode *p中， p是指向单链表中某个节点的指针变量， p是一个指针。



链表是什么？（单链表为例）

代码呈现

```
//-----单链表的存储结构-----
```

```
typedef struct LNode
```

```
{
```

```
ElemType data;
```

```
//节点的数据域 (🐶)
```

```
struct LNode *next;
```

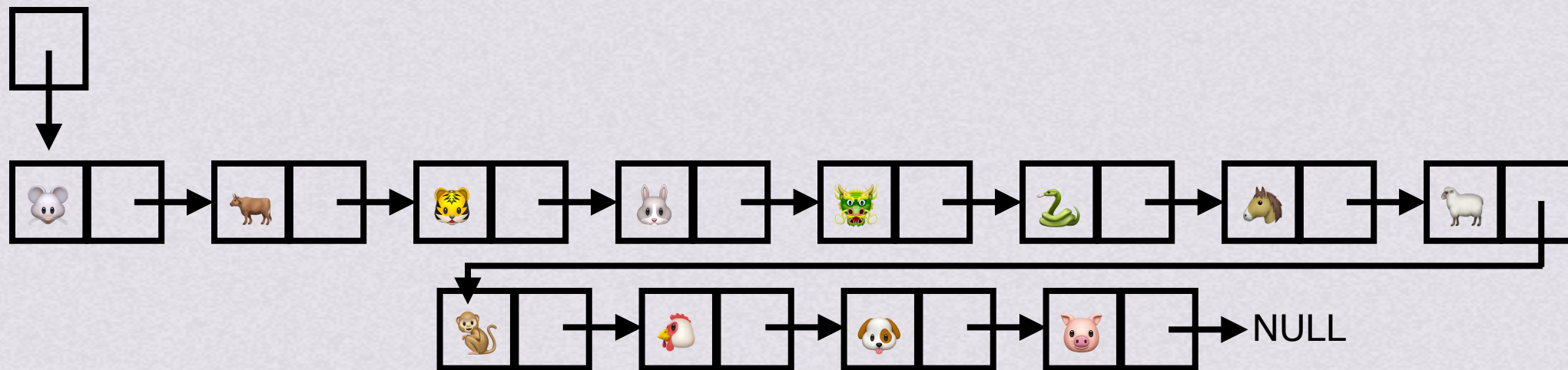
```
//节点的指针域（指向下一数据）
```

```
}LNode, *LinkList;
```



头结点的作用

头结点

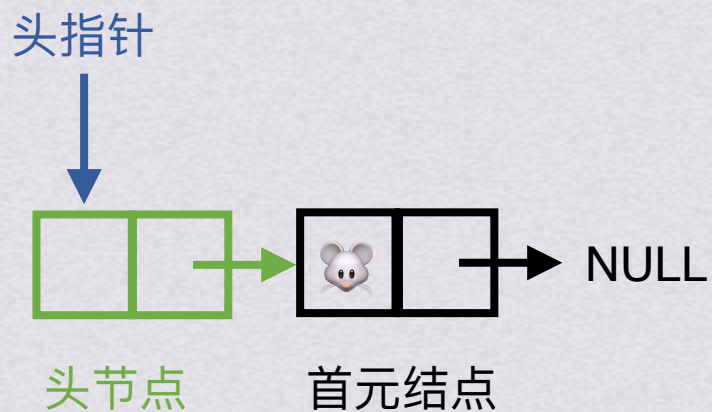




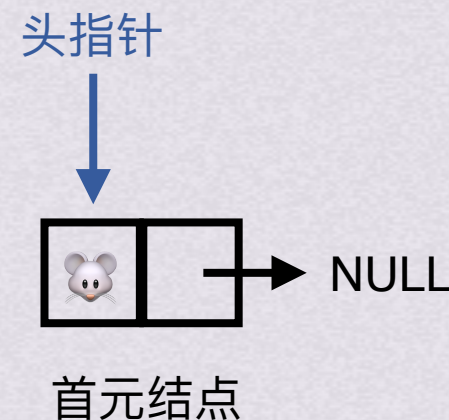
头结点的作用

一个链表无论有没有头结点，都一定有头指针！

如果增设头节点，那么头指针一定指向头节点；如果不增设头节点，那么头指针一定指向首元节点



有头节点的情况

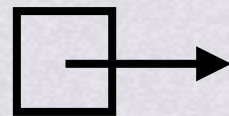


无头节点的情况



头结点的作用

头结点



- 1.便于首元结点的处理
- 2.便于空表和非空表的统一处理



基本操作的实现

单链表的初始化

```
Status InitList(LinkList &L)
{ //构造一个空的单链表L
  L=new LNode;           //生成新节点作为头结点，用头指针L指向头结点
  L->next=NULL;          //头结点的指针域置空
  return OK;
}
```



基本操作的实现

单链表按位取值

```
Status GetElem(LinkList L, int i, ElemType &e)
{
    // 在带头节点的单链表 L 中根据序号 i 获取元素的值, 用 e 返回 L 中第 i 个数据元素的值
    p=L->next; //初始化, p指向首元节点, 计数器j初值赋为1
    j=1;
    while(p && j<i){ //顺链域向后查找, 直到p为空或p指向第i个元素
        p=p->next; //p 指向下一个节点
        ++j; //计数器j相应加1
    }
    if(!p || j>i) return ERROR; //i值不合法 i>n 或 i<=0
    e=p->data; //取第i个节点的数据域
    return OK;
}
```

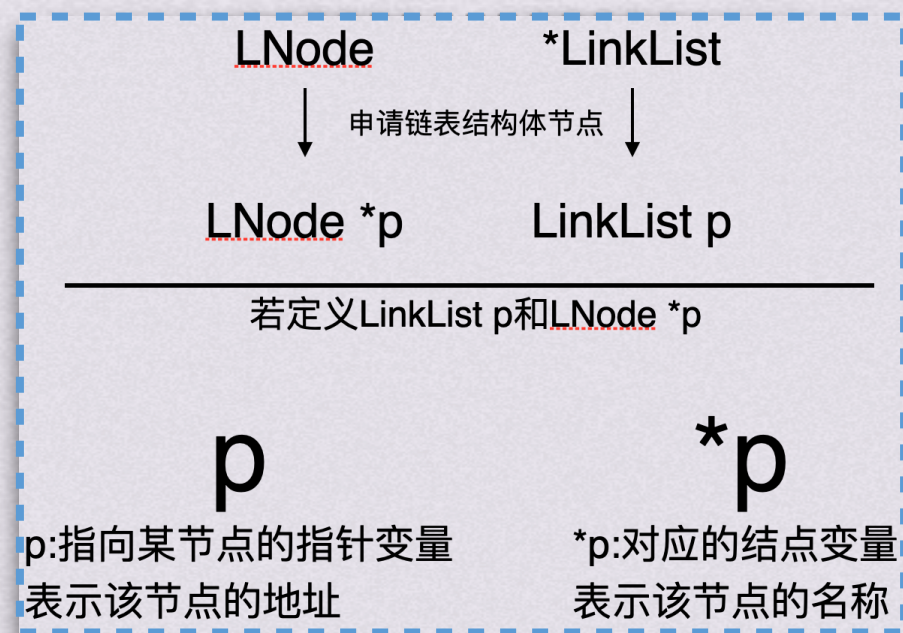



区别?

L.elem

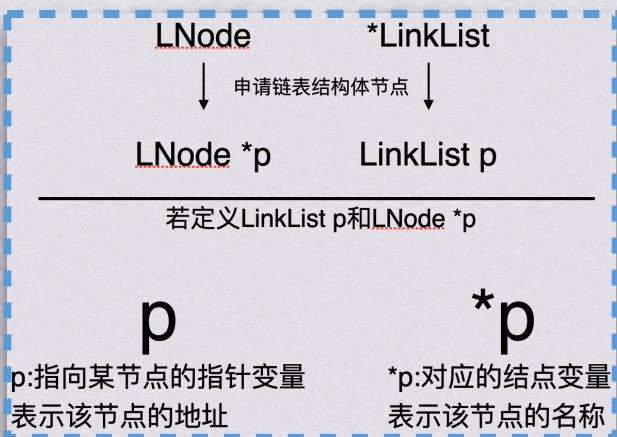
L->next

(*a).b等价于a→b





(*a).b等价于a→b



L.elem



申请时使用SqList L
(声明了一个SqList类型的结构体L)
(L是一个结构体变量)
用.来引用SqList结构体中定义的域

L->next



申请时使用LinkList L
(声明了一个指向LinkList结构体的指针L)
(L为指针)
用->来引用LinkList结构体中定义的域



总结来说，如果要使用■，只需要声明一个结构体

如果要使用→，需要声明一个结构体指针



基本操作的实现

单链表的按值查找

```
LNode *LocateElem(LinkList L, ElemType e)
{ // 在带头结点的单链表L中查找值为e的元素
    p = L->next;           // 初始化, p指向首元结点
    while(p && p->data != e) // 顺链域向后查找, 直到p为空或者p所指节点的数据域=e
        p = p->next;       // p指向下一个节点
    return p;              // 查找成功返回值为e的节点地址p, 查找失败p为NULL
}
```




基本操作的实现

单链表的插入

```
Status ListInsert(LinkList &L, int i, ElemType e)
{ // 在带头节点的单链表 L 中第 i 个位置插入值为 e 的新节点
    p=L; j=0;
    while(p && (j<i-1))
        {p=p->next; ++j;} // 查找第 i-1 个节点, p 指向该节点
    if(!p || j>i-1) return error; // i>n+1 或者 i<1
    s=new LNode; // 生成新节点*s
    s->data=e; // 将新节点*s 的数据域置为 e
    s->next=p->next; // 将节点*s 的指针域指向节点 ai
    p->next=s; // 将节点*p 的指针域指向节点*s
    return ok;
}
```



```
Status ListDelete(LinkList &L, int i)
{ // 在带头节点的单链表 L 中, 删除第 i 个元素
    p=L; j=0;
    while((p->next) && (j<i-1)) // 查找第 i-1 个节点, p 指向该节点
        {p=p->next; ++j;}
    if(!(p->next) || (j>i-1)) return error; // 当 i>n 或 i<1 时, 删除位置不合理
    q=p->next; // 临时保存被删除节点的地址以备释放
    p->next=q->next; // 改变删除节点前驱节点的指针域
    delete q; // 释放删除节点的空间
    return ok;
}
```




插入/删除第*i*个节点，时间复杂度是多少？

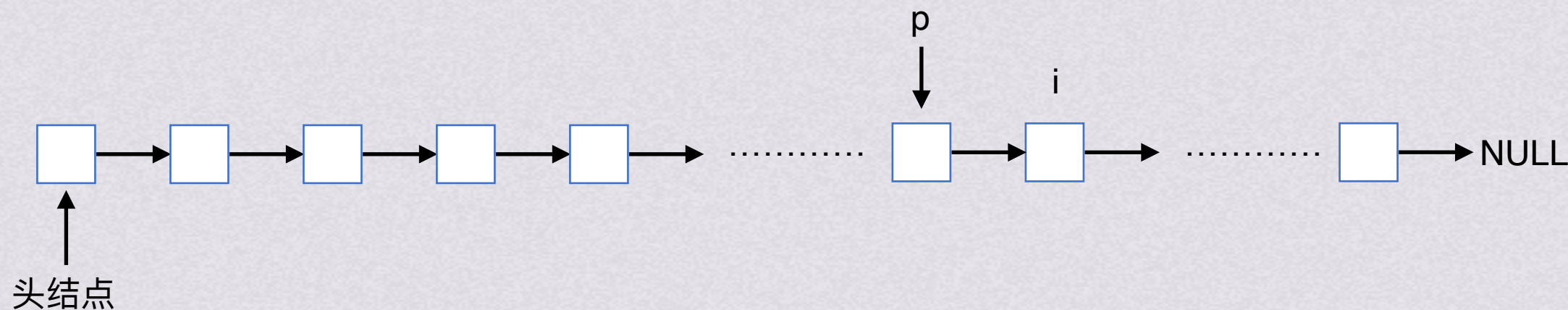
答案： $O(n)$



插入/删除第i个节点，时间复杂度是多少？

答案：O (n)

链表想找到第i个位置，只能通过遍历来实现



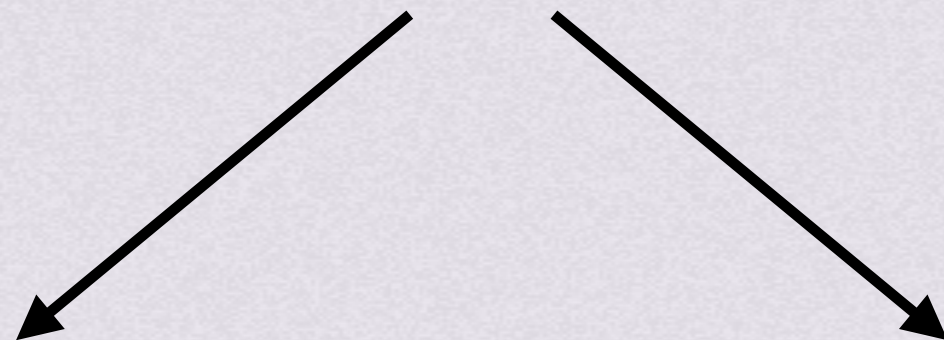
遍历寻找前驱产生了复杂度



创建单链表

前插法

后插法





前插法

代码实现

```
void CreateList_H(LinkList &L, int n)
{ // 逆位序输入n个元素的值，建立带表头节点的单链表L
  L = new LNode;
  L->next = NULL; // 建立一个带头结点的单链表
  for (i = 0; i < n; ++i) {
    p = new LNode; // 生成新节点*p
    cin >> p->data; // 输入元素值交给新节点*p的数据域
    p->next = L->next; // 将新节点*p插入到头结点之后
    L->next = p;
  }
}
```



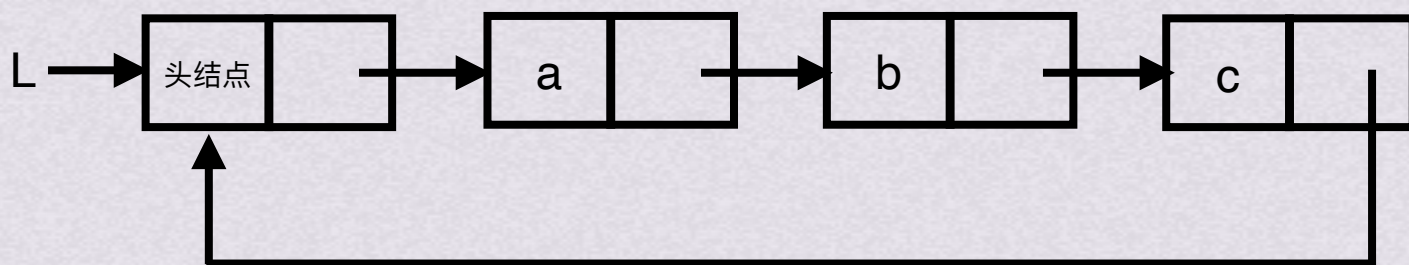

尾插法

代码实现

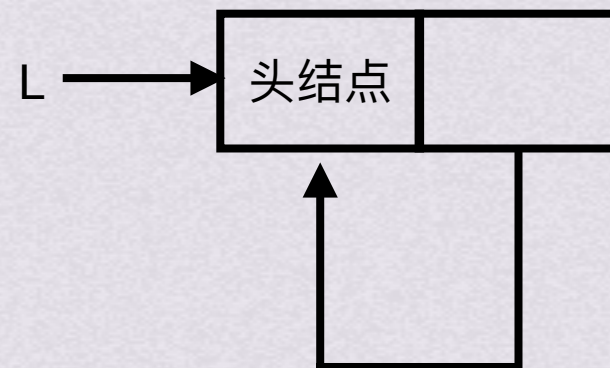
```
void CreateList_R(LinkList &L, int n)
{ // 正位序输入n个元素的值，建立带表头节点的单链表L
    L = new LNode;
    L->next = NULL; // 建立一个带头结点的单链表
    r = L;           // 尾指针r指向头结点
    for (i = 0; i < n; ++i) {
        p = new LNode; // 生成新节点
        cin >> p->data; // 输入元素值赋给新节点*p的数据域
        p->next = NULL; r->next = p; // 将新节点*p插入尾节点*r之后
        r = p; // r指向新的尾节点*p
    }
}
```



循环链表



(非空情况)



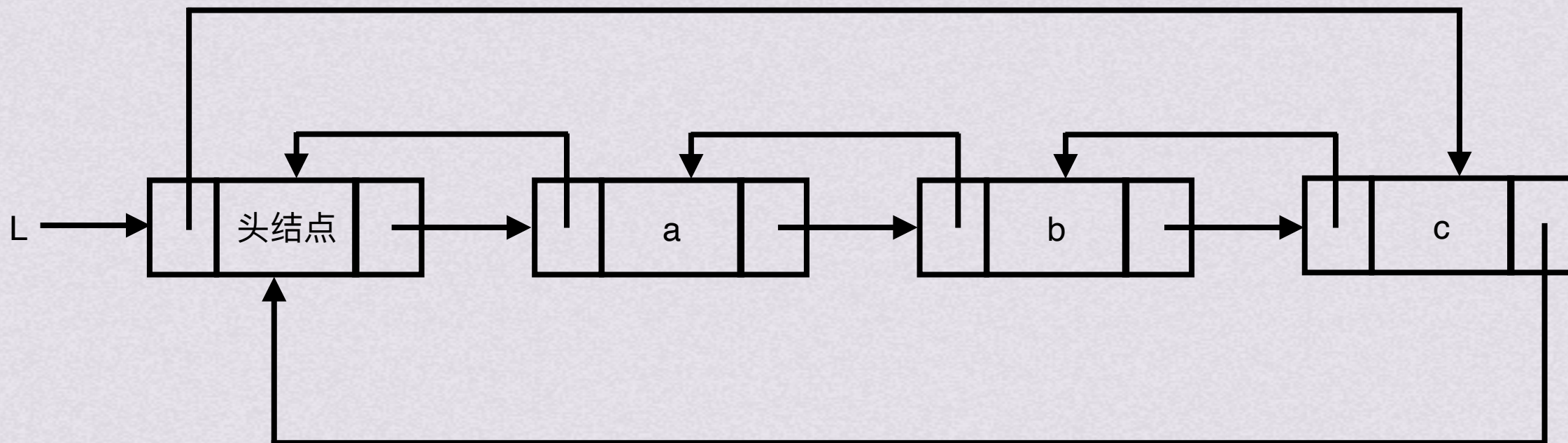
(空表情况)

优点： 1.任何节点都可以作为头结点

2.可以用于实现队列。这一点408真题是作为应用题考查过的，但难度对于目前阶段的大家来说会比较高，我们先let it go。



双向链表



解决了单链表的痛点：插入/删除操作因为遍历寻找前驱而造成的时间复杂度



双向链表

存储结构的代码表示

```
//-----双向链表的存储结构-----  
typedef struct DuLNode{  
    ElemType data;                //数据域  
    struct DuLNode *prior;        //指向直接前驱  
    struct DuLNode *next;         //指向直接后继  
}DuLNode, *DuLinkList;
```




双向链表

插入操作

```
Status ListInsert_DuL(DuLinkList &L, int i, ElemType e)
{
    // 在带头节点的双向链表 L 中第 i 个位置之前插入元素 e
    if(! (p=GetElem_DuL(L, i)))                //在L中确定第i个元素的位置指针p
        return ERROR;                          //p为NULL时, 第i个元素不存在
    s=new DuLNode;                               //生成新节点*s
    s->data=e;                                   //将节点*s数据域置为e
    s->prior=p->prior; p->prior->next=s; s->next=p; //将节点*s插入L中
    p->prior=s;
    return OK;
}
```



删除操作

```
Status ListDelete_DuL(DuLinkList &L, int i)
{
    // 删除带头节点的双向链表 L 中的第 i 个元素
    if (! (p = GetElem_DuL(L, i))) // 在L中确定第i个元素的位置指针p
        return ERROR; // p为NULL时, 第i个元素不存在
    p->prior->next = p->next; // 修改被删节点的前驱节点的后继指针
    p->next->prior = p->prior; // 修改被删节点的后继节点的前驱指针
    delete p; // 释放被删节点的空间
    return OK;
}
```




```
4 struct person {  
5     char name;  
6     char age;  
7     char sex;  
8 };  
9  
10 int main() {  
11     struct person p;  
12     struct person *q;  
13  
14     q = (struct person *)malloc(sizeof(struct person));  
15  
16     p.name = 1;  
17     p.age = 2;  
18     p.sex = 4;  
19  
20     q->name = 1;  
21     q->age = 2;  
22     q->sex = 3;  
23  
24     printf("p = %06x\n", p);  
25     printf("*q = %06x\n", *q);  
26     printf("q = %06x", q);  
27     free(q);  
28  
29     return 0;  
30 }
```

关于*p

```
p = 040201  
*q = 030201  
q = 1b67260
```